

Chapter 4

Network Layer: The Data Plane

A note on the use of these Powerpoint slides:

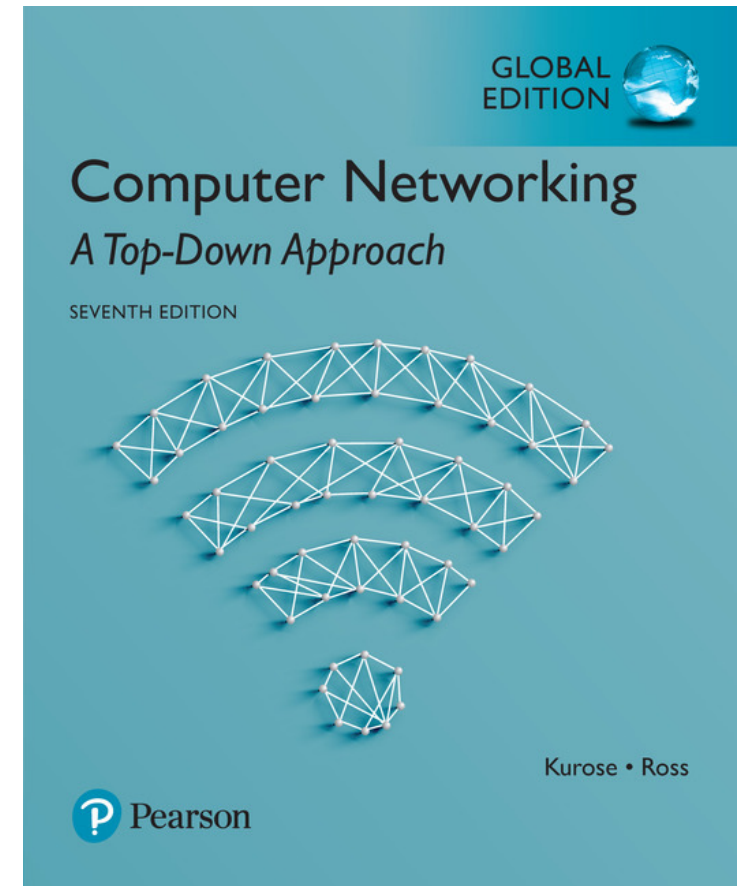
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

© All material copyright 1996-2016
J.F Kurose and K.W. Ross, All Rights Reserved

Modified by Man Hon Cheung



Computer Networking: A Top Down Approach

7th Edition, Global Edition
Jim Kurose, Keith Ross
Pearson
April 2016

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

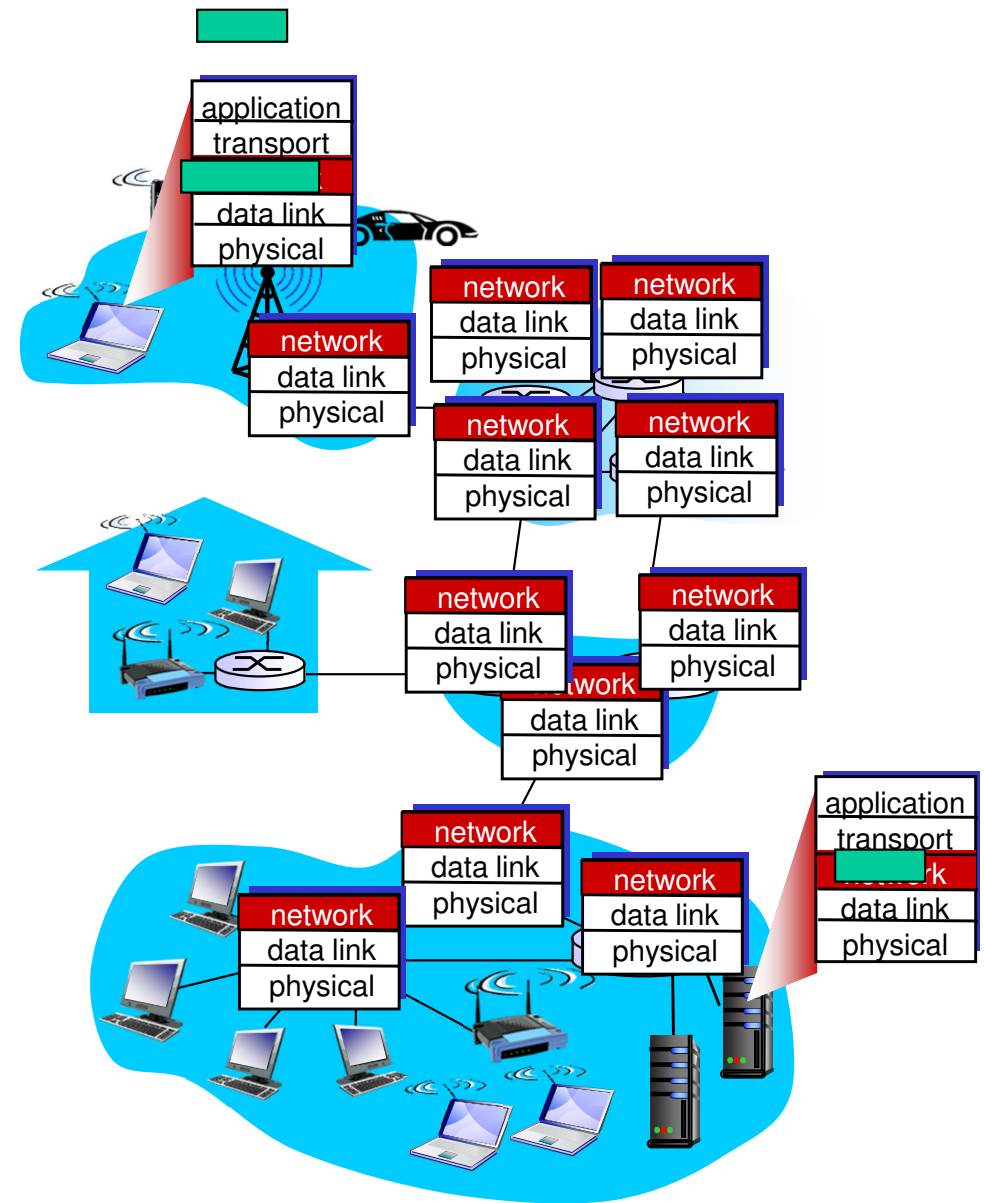
Chapter 4: network layer

chapter goals:

- understand principles behind network layer services, focusing on data plane:
 - network layer service models
 - forwarding versus routing
 - how a router works
 - generalized forwarding
- instantiation, implementation in the Internet

Network layer

- transport segment from sending to receiving host
 - on sending side encapsulates segments into datagrams
 - on receiving side, delivers segments to transport layer
- network layer protocols in every host and router
 - router examines header fields in all IP datagrams passing through it



Two key network-layer functions

- forwarding: ^{local} move packets from router's input to appropriate router output

- routing: ^{global} determine route taken by packets from source to dest.

analogy:

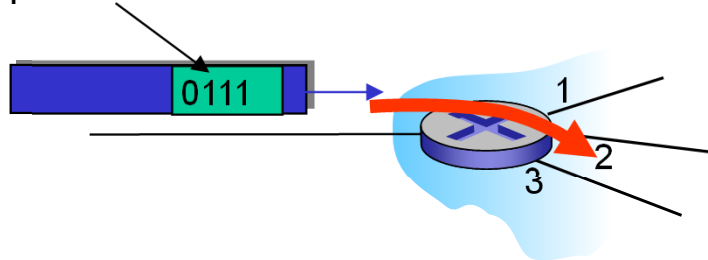
- ❖ *forwarding*: process of getting through single interchange
- ❖ *routing*: process of planning trip from source to dest

Network layer: data plane, control plane

Cha.4 Data plane (forwarding) Cha.5 Control plane (routing)

- local, per-router function
- determines how datagram arriving on router input port is forwarded to router output port
- forwarding function

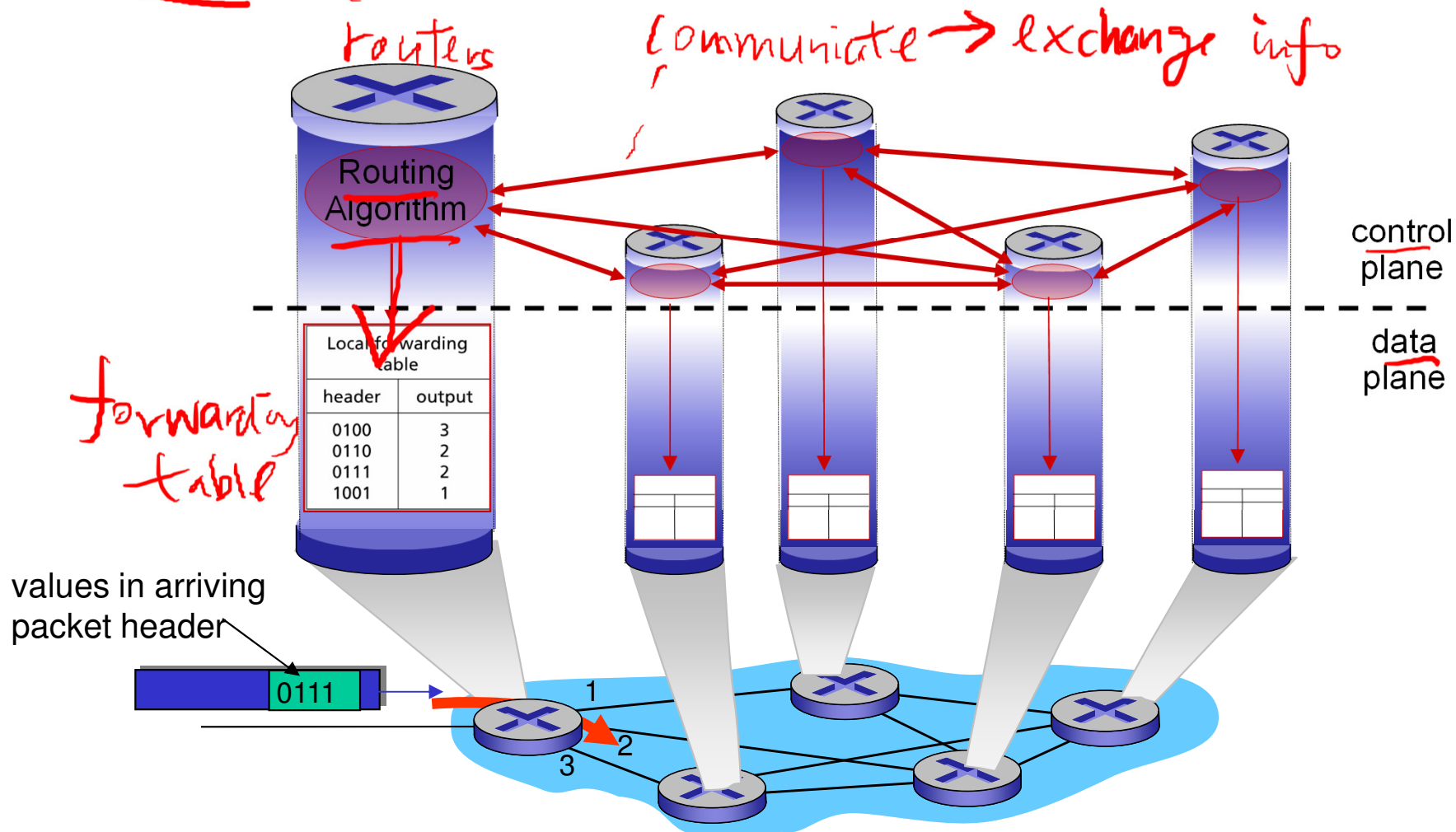
values in arriving packet header



- network-wide logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
 - traditional routing algorithms: implemented in routers
 - software-defined networking (SDN): implemented in (remote) servers

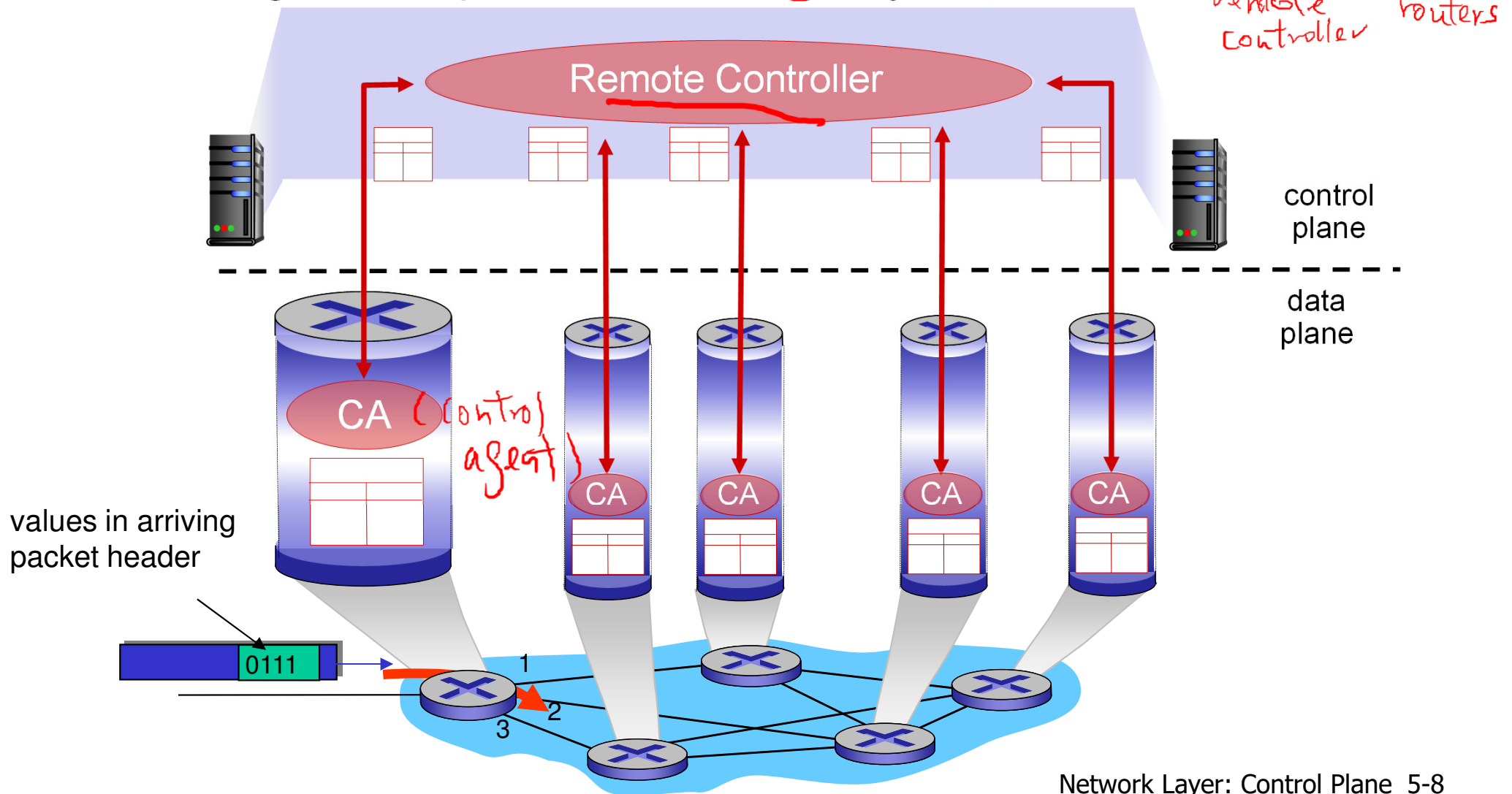
Per-router control plane (Traditional routing)

Individual routing algorithm components *in each and every router* interact in the control plane



Logically centralized control plane SDN

- Remote controller computes and distributes the forwarding tables to be used by each and every router.
- Routing devices perform forwarding only.



Network service model

Q: What *service model* for “channel” transporting datagrams from sender to receiver?

example services for
individual datagrams:

- ❖ guaranteed delivery
- ❖ guaranteed delivery with less than 40 msec delay

example services for a flow
of datagrams:

- in-order datagram delivery
- guaranteed minimum bandwidth to flow
- restrictions on changes in inter-packet spacing

Network layer service models:

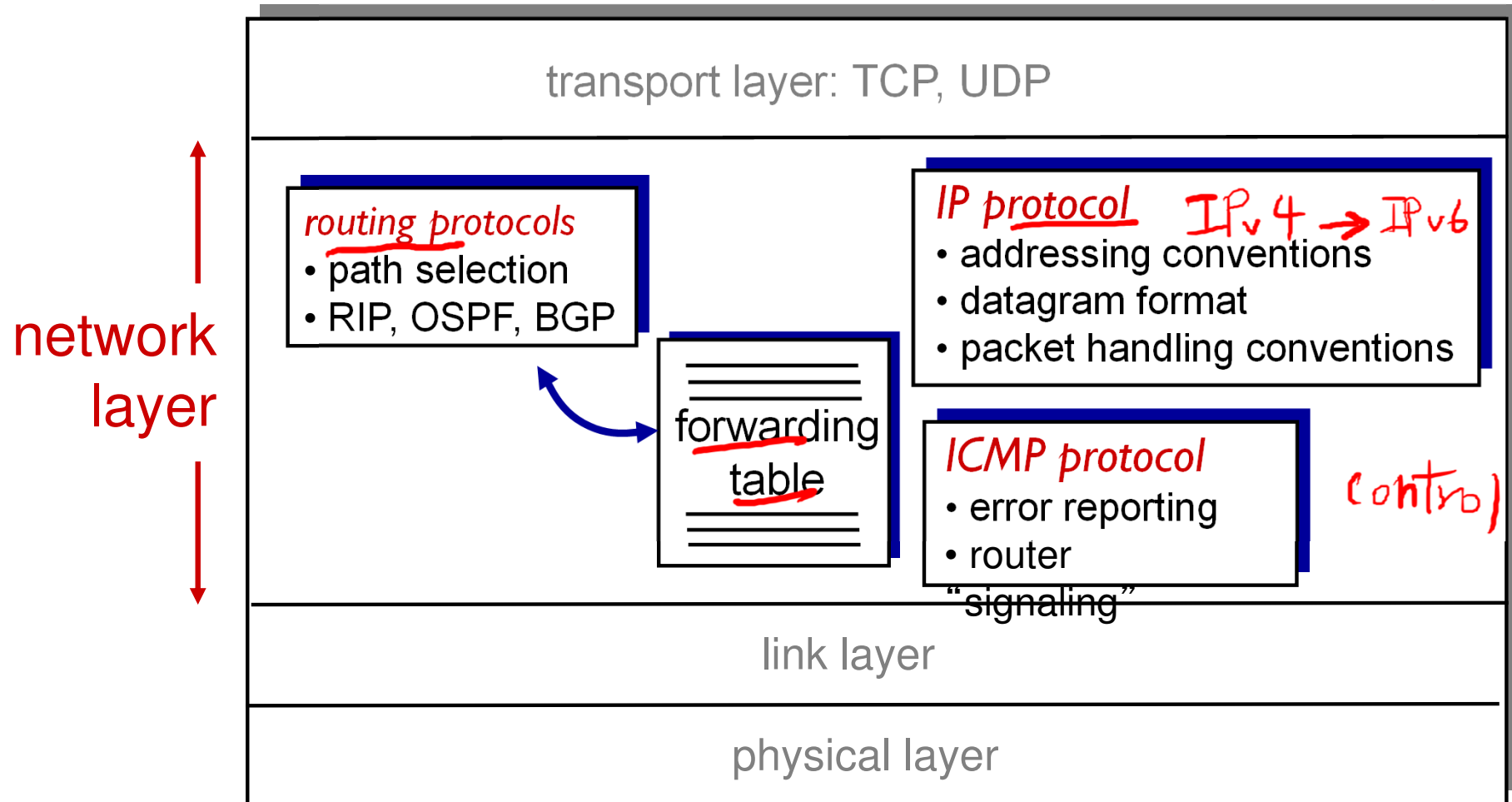
Network Architecture	Service Model	Guarantees ?				Congestion feedback
		Bandwidth	Loss	Order	Timing	
→ Internet	"best effort"	none	no	no	no	no (inferred via loss)
ATM	Constant bit rate (CBR)	constant rate	yes	yes	yes	no congestion
ATM	Available bit rate (ABR)	guaranteed minimum	no	yes	no	yes

unreliable Internet!

The Internet network layer

host, router network layer functions:

Roadmap



Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

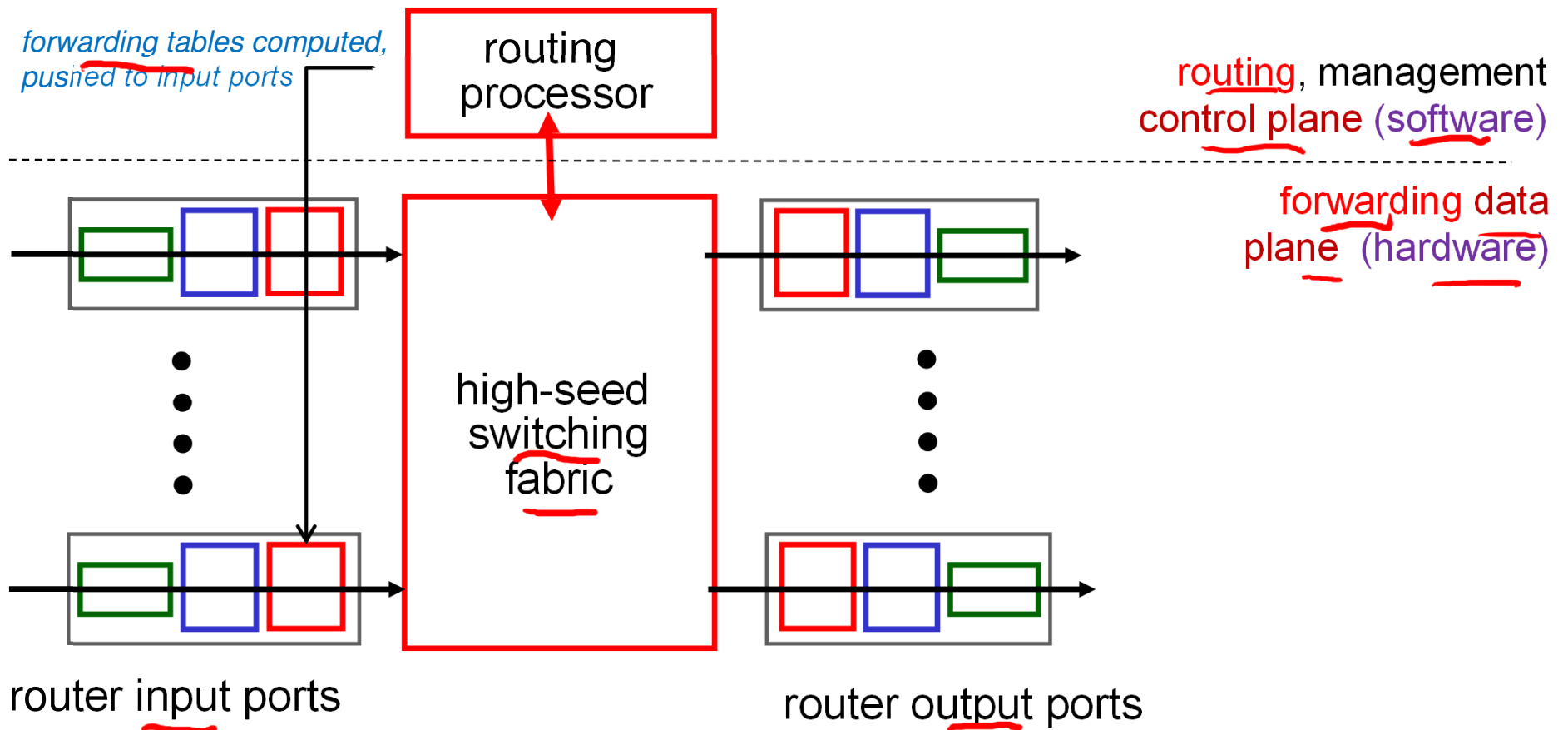
4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

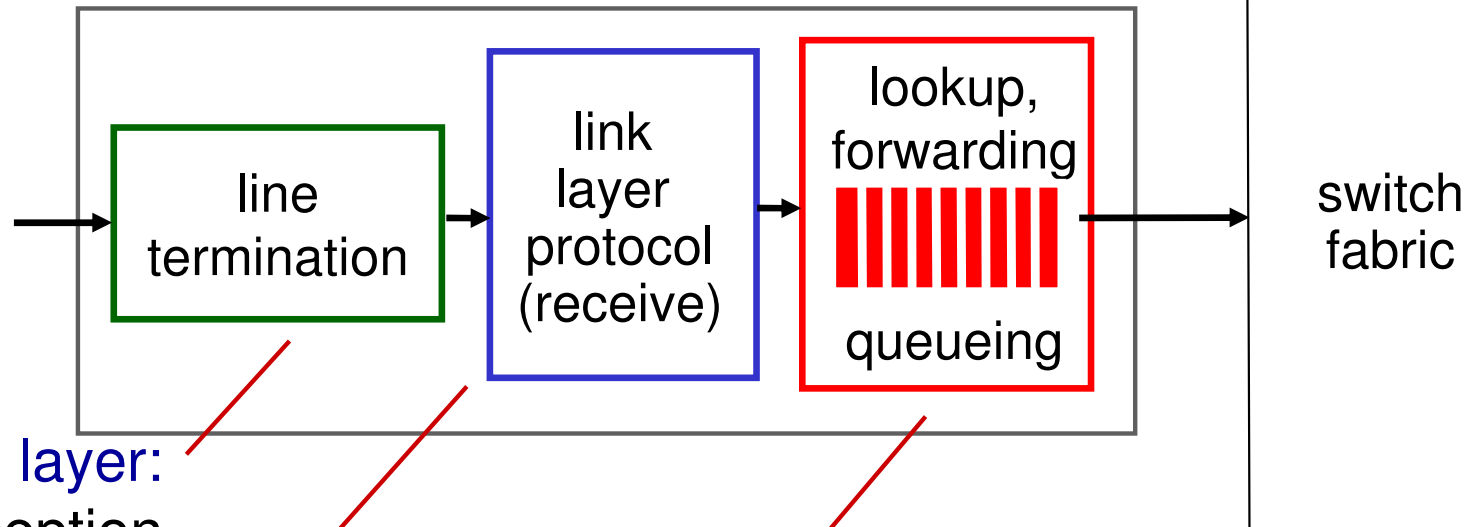
Router architecture overview

two key router functions:

- ❖ run **routing** algorithms/protocol (RIP, OSPF, BGP)
- ❖ **forward** datagrams from incoming to outgoing link



Input port functions



physical layer:
bit-level reception

data link layer:
e.g., Ethernet
see chapter 6

decentralized switching:

- given datagram dest., lookup output port using forwarding table in input port memory (*“match plus action”*)
- **destination-based forwarding**: forward based only on destination IP address (traditional)
- **generalized forwarding**: forward based on any set of header field values

Datagram forwarding table

Destination Address Range (IPv4)	Link Interface
prefix 11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Longest prefix matching

longest prefix matching

when looking for forwarding table entry for given destination address, use longest address prefix that matches destination address.

Destination Address Range	Link interface
^{prefix} 11001000 00010111 00010*** *****	A 0
11001000 00010111 00011000 *****	B 1
11001000 00010111 00011*** *****	C 2
otherwise	D 3

examples:

*dest
address*

DA: 11001000 00010111 00010110 10100001

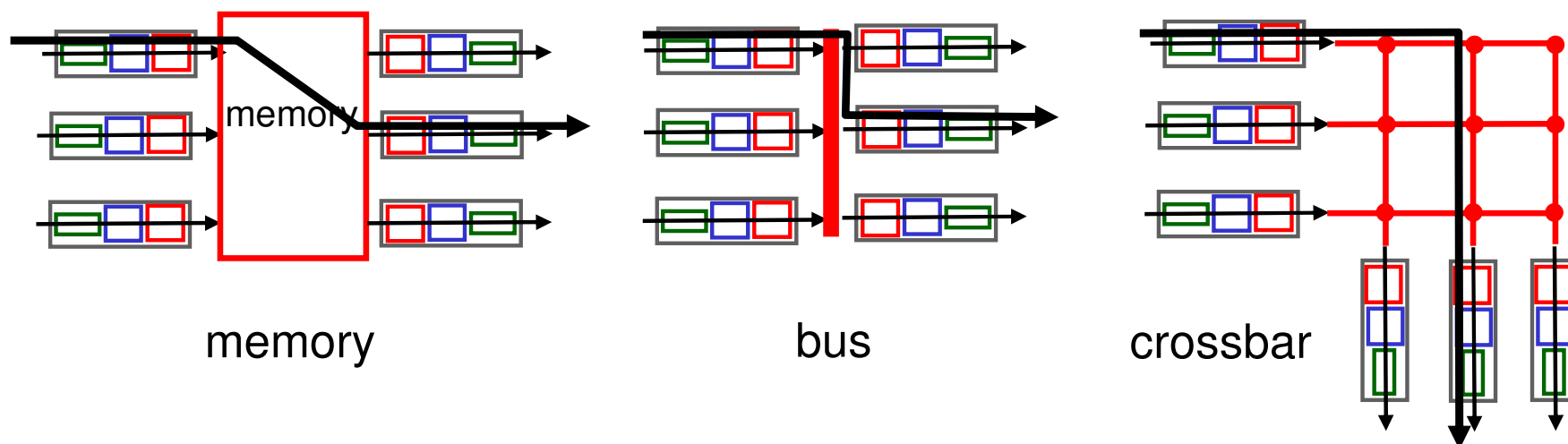
which interface? **0**

DA: 11001000 00010111 00011000 10101010

which interface? **1**

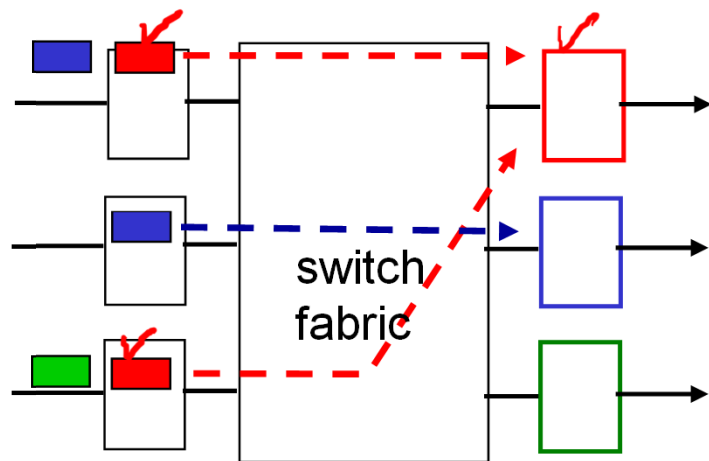
Switching fabrics

- ❖ transfer packet from input buffer to appropriate output buffer
- ❖ switching rate: rate at which packets can be transfer from inputs to outputs
 - often measured as multiple of input/output line rate
- ❖ three types of switching fabrics...



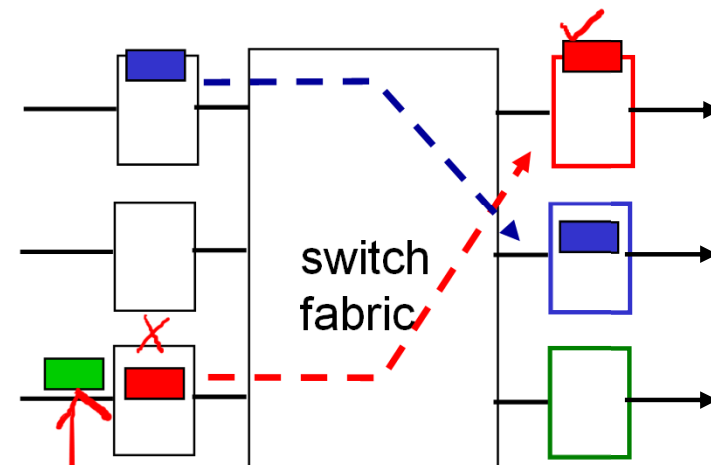
Input port queuing

- fabric slower than input ports combined -> queueing may occur at input queues
 - queueing delay and loss due to input buffer overflow!
- Head-of-the-Line (HOL) blocking: queued datagram at front of queue prevents others in queue from moving forward



output port contention:
only one red datagram can be
transferred.

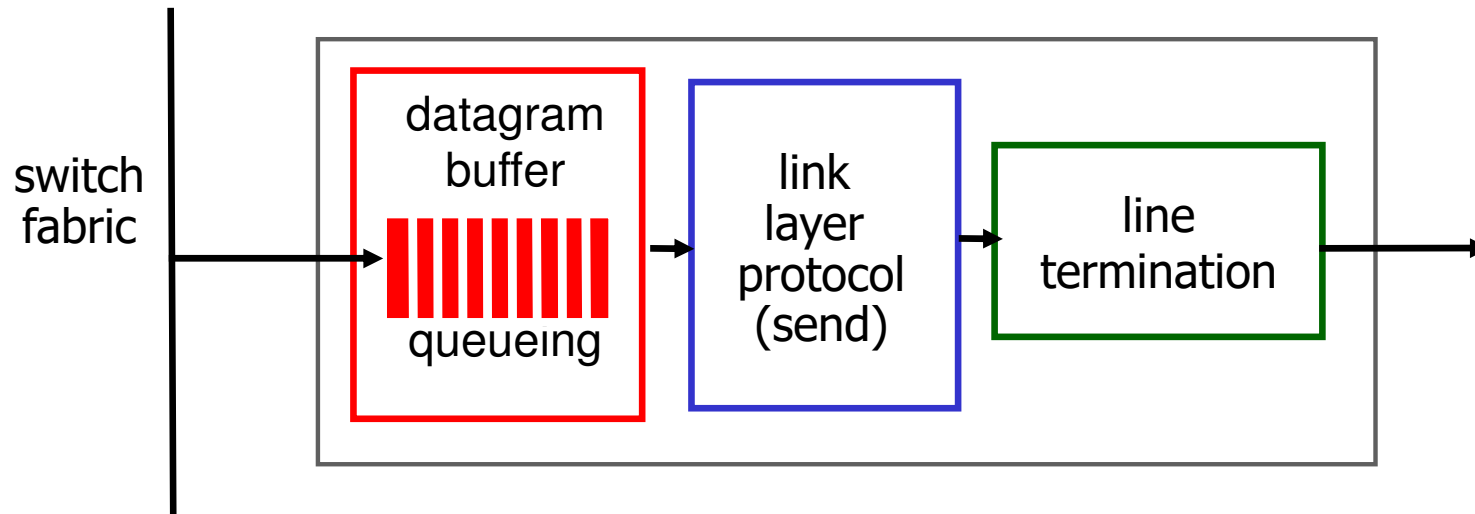
lower red packet is blocked



one packet time later:

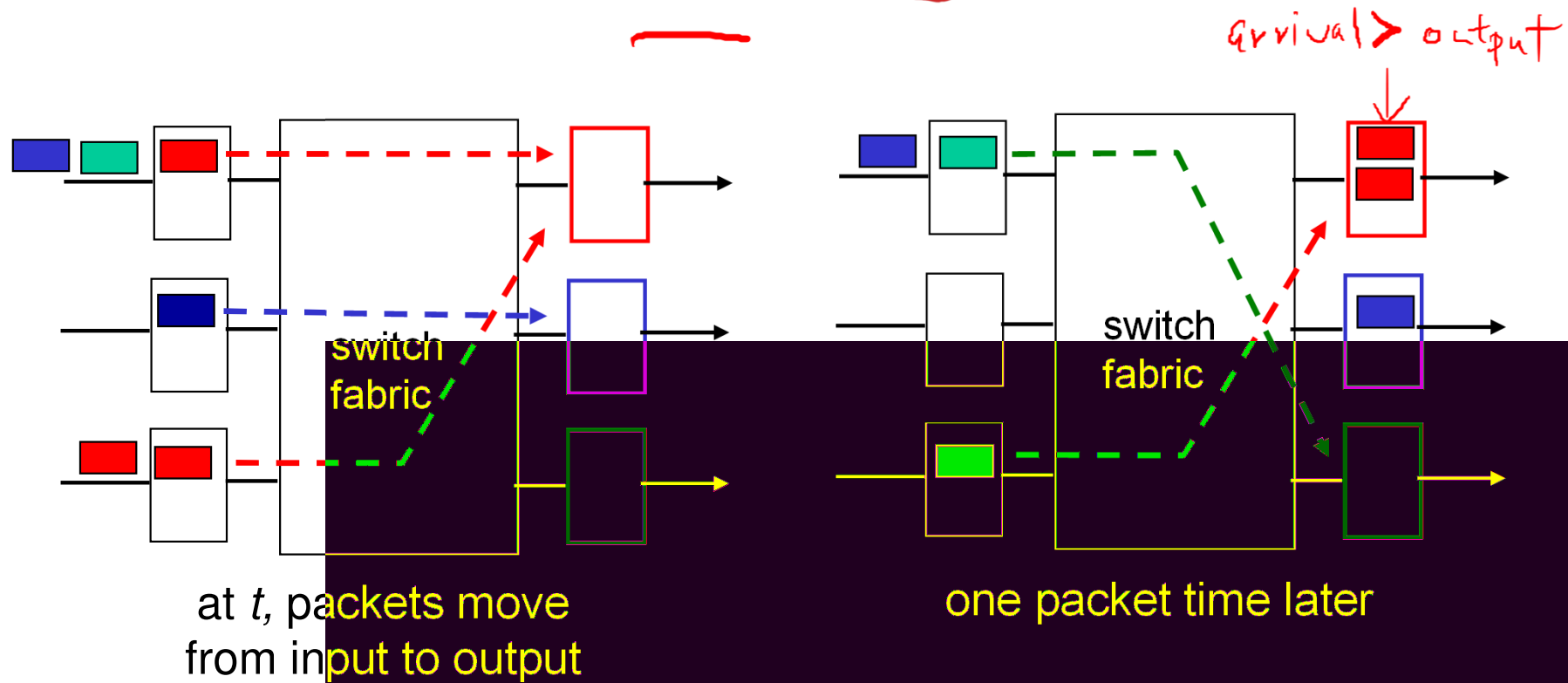
green packet
experiences HOL
blocking

Output ports



- ❖ *buffering* required when datagrams arrive from fabric faster than the transmission rate
 - ❖ Datagram (packets) can be lost due to congestion, lack of buffers
- ❖ *scheduling discipline* chooses among queued datagrams for transmission
 - ❖ Priority scheduling vs network neutrality

Output port queueing



- buffering when arrival rate via switch exceeds output line speed
- queueing (delay) and loss due to output port buffer overflow!

How much buffering?

$$\begin{aligned} B &= RTT \times C \\ &= 0.25 \text{ sec} \times 10 \text{ Gbps} \\ &= 2.5 \text{ Gbits} \end{aligned}$$

- RFC 3439 rule of thumb: average buffering equal to “typical” RTT (say 250 msec) times link capacity C
 - e.g., C = 10 Gbps link: 2.5 Gbit buffer
- recent recommendation: with N TCP flows passing through a link, amount of buffering needed is equal to
$$\frac{RTT \cdot C}{\sqrt{N}}$$

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

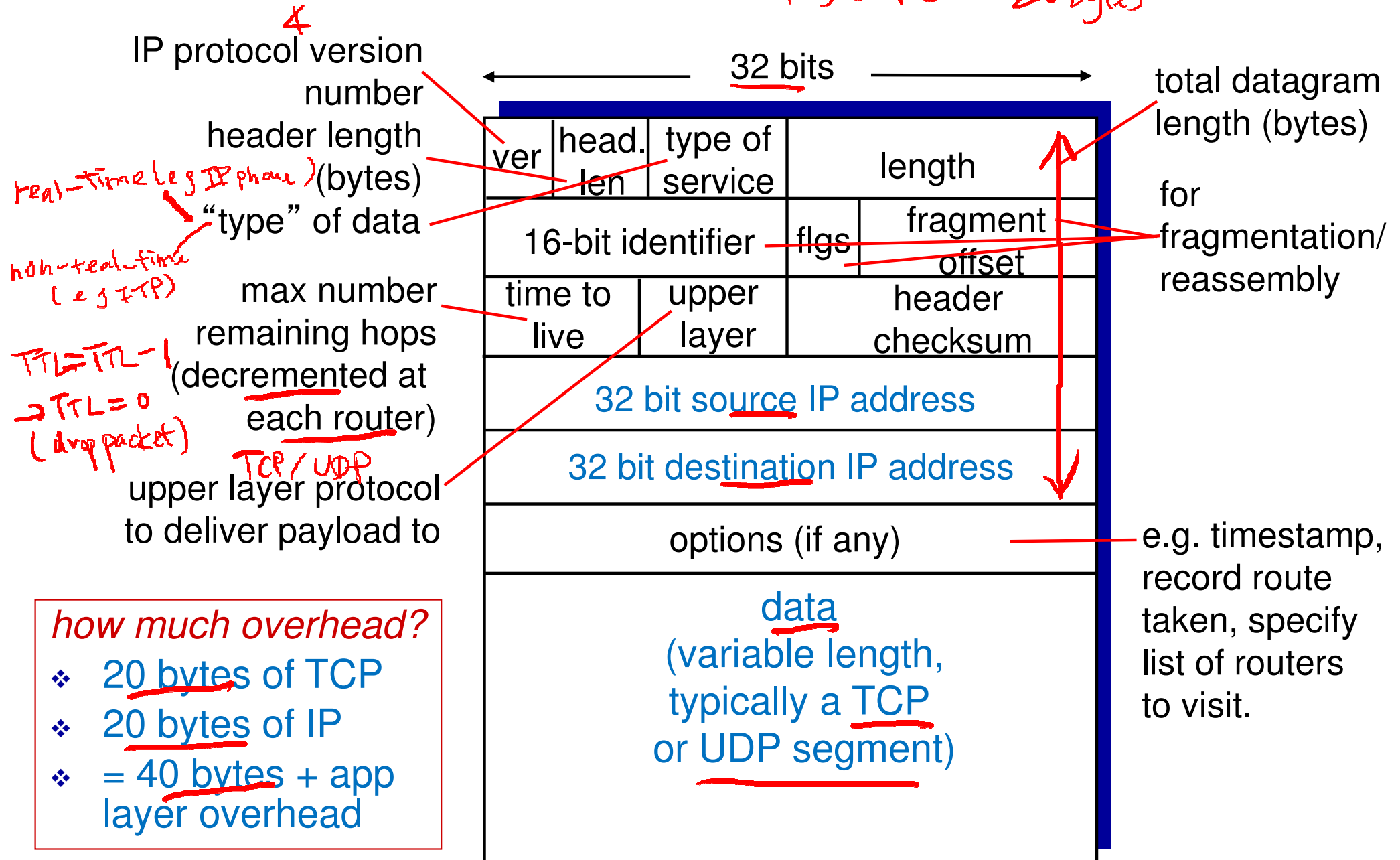
- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

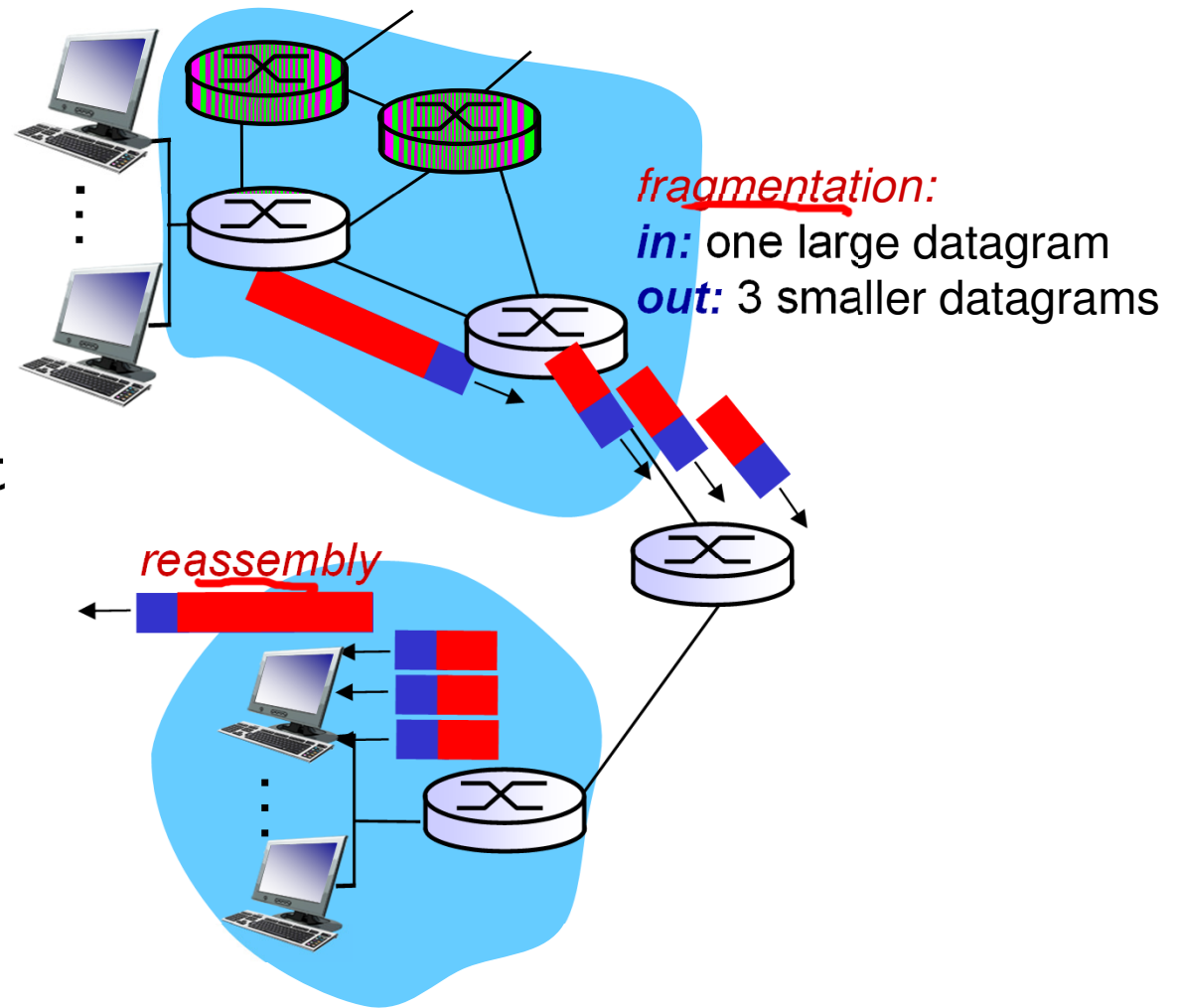
IP datagram format

*min IP header
= 4 bytes * 5 = 20 bytes*



IP fragmentation, reassembly

- **Maximum transmission unit (MTU):** largest possible **link-layer** frame
 - different link types, different MTUs
- large IP datagram divided ("fragmented") within net
 - one datagram becomes several datagrams
- "reassembled" only at final destination:
 - IP header bits used to identify, order related fragments



IP fragmentation, reassembly

example:

- ❖ 4000 byte datagram
- ❖ MTU = 1500 bytes

max "data" in one smaller datagram
 $= 1500 - 20 = 1480$
 1480 bytes in
 data field

offset =
 $1480 / 8 = 185$

length	ID	fragflag	offset	data
=4000	=x	=0	=0	= 4000 - 20 = 3980

one large datagram becomes
 several smaller datagrams

length	ID	fragflag	offset	data
=1500	=x	=1	=0	1480

length	ID	fragflag	offset	data
=1500	=x	=1	=185	1480

length	ID	fragflag	offset	data
=1040	=x	=0	=370	1020



IP fragmentation, reassembly (cont'd)

Fragment	Bytes	ID	Offset	Flag
1st fragment	1,480 bytes in the data field of the IP datagram	identification = 777	offset = 0 (meaning the data should be inserted beginning at byte 0)	flag = 1 (meaning there is more)
2nd fragment	1,480 bytes of data	identification = 777	offset = 185 (meaning the data should be inserted beginning at byte 1,480. Note that $185 \cdot 8 = 1,480$)	flag = 1 (meaning there is more)
3rd fragment	1,020 bytes (= 3,980-1,480-1,480) of data	identification = 777	offset = 370 (meaning the data should be inserted beginning at byte 2,960. Note that $370 \cdot 8 = 2,960$)	flag = 0 (meaning this is the last fragment)

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

IPv4 Network Addresses

❖ IPv4 Address

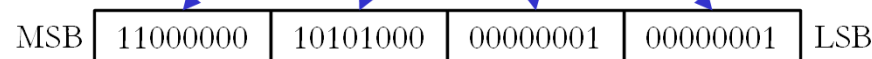
- The current addressing standard used in the Internet

- 32 bits

- Common representations:

- Dot notation: 192.168.1.1

- Binary:



- Address space: 4,294,967,296 = 2^{32}

- USA (35.9%, 4.9 IP addresses per person) #
- Hong Kong (0.3%, 1.6 IP addresses per person) #
- China (7.7%, 0.25 IP addresses per person) #

Source: https://en.wikipedia.org/wiki/List_of_countries_by_IPv4_address_allocation

→ IPv6

→ 128 bits

→ 2^{128} IoT

Classful IP Address (obsolete)

❖ Hierarchical Architecture

- An IP address is divided into a network and a host part.
- Compare this to telephone numbers (area code + number)
- The original hierarchy for IP addresses (now obsolete!)

	netw		host		
<u>Class A IP Address:</u>	01010101	10101000	00000001	00000001	For <u>large</u> organizations
<u>Class B IP Address:</u>	10010011	10101000	00000001	00000001	For medium organizations
<u>Class C IP Address:</u>	11000101	10101000	00000001	00000001	For <u>small</u> organizations
<u>Class D IP Address:</u>	11100101	10101000	00000001	00000001	Multicast addresses (local network)
<u>Class E IP Address:</u>	11110000	10101000	00000001	00000001	For research purposes

Note: The number of network bits is implicit to a given IP address via prefix coding.

Classful IP Address

❖ Subnet Mask

- A bit mask for computing the network address from an IP address
 - “255.255.0.0” or “/16” for a class B subnet mask
- Network Address Computation

Given

IP address (decimal):

137	189	11	73
-----	-----	----	----

IP address (binary):

10001001	10111101	00001011	01001001
----------	----------	----------	----------

Subnet mask (decimal):

255	255	0	0
-----	-----	---	---

Subnet mask (binary):

11111111	11111111	00000000	00000000
----------	----------	----------	----------

Network address (B):

10001001	10111101	00000000	00000000
----------	----------	----------	----------

Network address (D):

137	189	0	0
-----	-----	---	---

Broadcast address (D):

137	189	255	255
-----	-----	-----	-----

The prefix bits “10” tell this is a Class B address.

IP Address bitwise-AND subnet mask

The host bits are all set to 0.

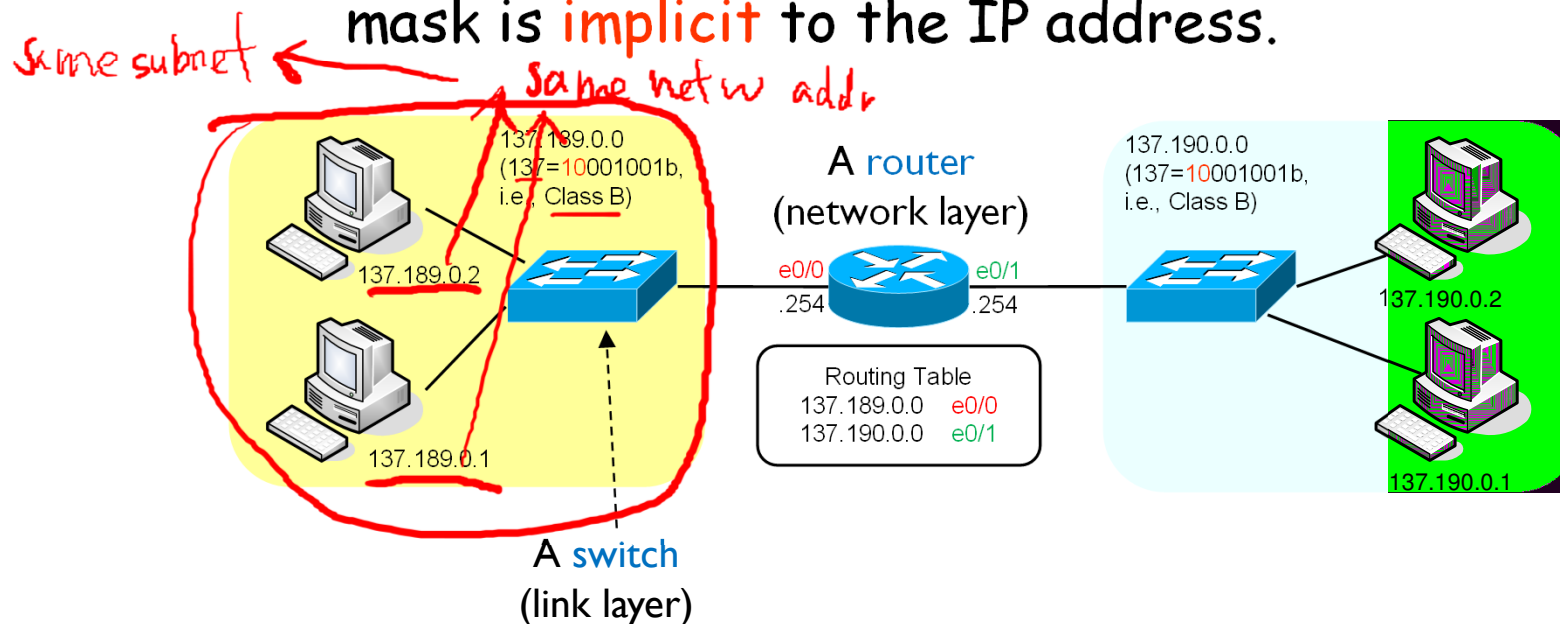
The host bits are all set to 1.

Note: No host can use host address with all bits zero (‘this network’) or all bits one (‘broadcast’).

Classful Routing

❖ Relation to Routing/Forwarding

- Hosts within the **same subnet** can reach one another directly via **link layer**, i.e., no need for routing.
- Routers forward packets using routing table which maps **ranges of IP addresses** to an output interface
 - The **network address part** in classful IP addresses serves as **the range indicator**. Note that the subnet mask is **implicit** to the IP address.



Classful Routing

❖ Issues with **Classful** IP Addresses:

- A **Class C** network is often **too small** for a company (max 254 hosts) but a Class B network is likely too large (max 65534 hosts) $2^{16} - 2$
- **Class B and Class A** networks are **not efficiently utilized**, i.e., only a small subset of the address space is used by hosts, the rest wasted.
- **Not enough flexibility in address allocation!**

Classless IP Address

- ❖ **Classless Inter-Domain Routing (CIDR)**
 - Remove the limitation of fixed number of network address bits in classful IP addresses
 - Class prefix no longer represents the number of network address bits
 - The size of the network address part is explicitly indicated by a subnet mask, e.g., 255.255.128.0 or a length suffix, e.g., /17.
 - Unlike classful routing, subnet masks will need to be exchanged between routers.

Classless IP Address

❖ Classless Inter-Domain Routing (CIDR)

- Example: 137.189.178.73 /17

IP addr 17 bit subnet mask

Given

IP address (decimal):	137	189	178	73
IP address (binary):	10001001	10111101	10110010	01001001
Subnet mask (decimal):	255	255	128	0
Subnet mask (binary):	11111111	11111111	10000000	00000000
Network address (B):	10001001	10111101	10000000	00000000
Network address (D):	137	189	128	0
Broadcast address (D):	137	189	255	255

AND

17 bits (network part) 15 bits (host part)

IP addresses: how to get one?

Q: How does a *host* get IP address?

- ❖ hard-coded by system admin in a file
 - Windows: control-panel->network->configuration->tcp/ip->properties
 - UNIX: /etc/rc.config
- ❖ DHCP: Dynamic Host Configuration Protocol:
dynamically get address from as server
 - “plug-and-play”

DHCP: Dynamic Host Configuration Protocol

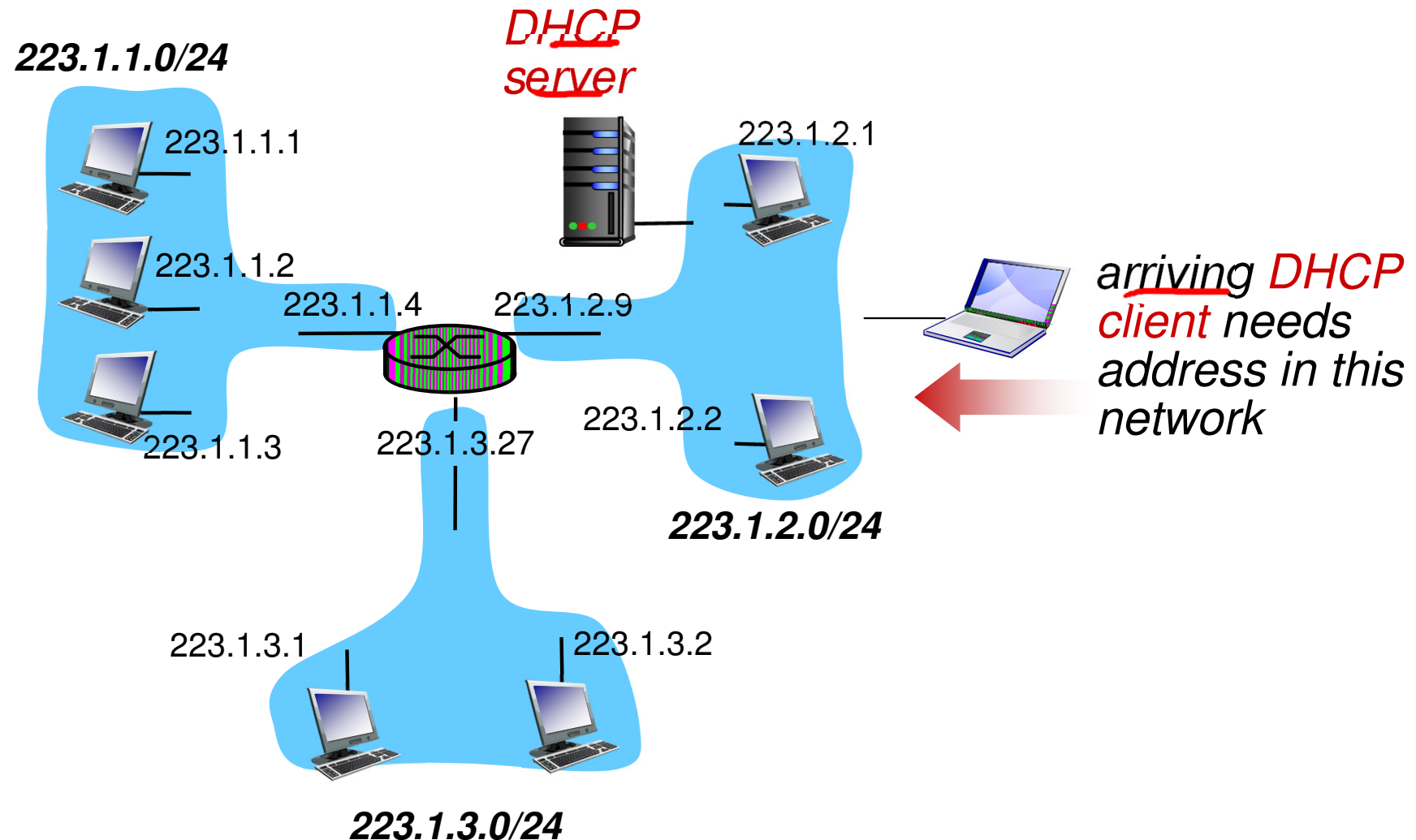
goal: allow host to dynamically obtain its IP address from network server when it joins network

- can renew its lease on address in use
- allows reuse of addresses (only hold address while connected/“on”)
- support for mobile users who want to join network (more shortly)

DHCP overview:

- host broadcasts “DHCP discover” msg
- DHCP server responds with “DHCP offer” msg
- host requests IP address: “DHCP request” msg
- DHCP server sends address: “DHCP ack” msg

DHCP client-server scenario

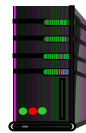


DHCP client-server scenario

DHCP server: 223.1.2.5

DHCP discover

arriving
client



Broadcast: is there a
DHCP server out there?

DHCP offer

Broadcast: I'm a DHCP
server! Here's an IP
address you can use

DHCP request

Broadcast: OK. I'll take
that IP address!

// yiaddr: IP address for client

DHCP ACK

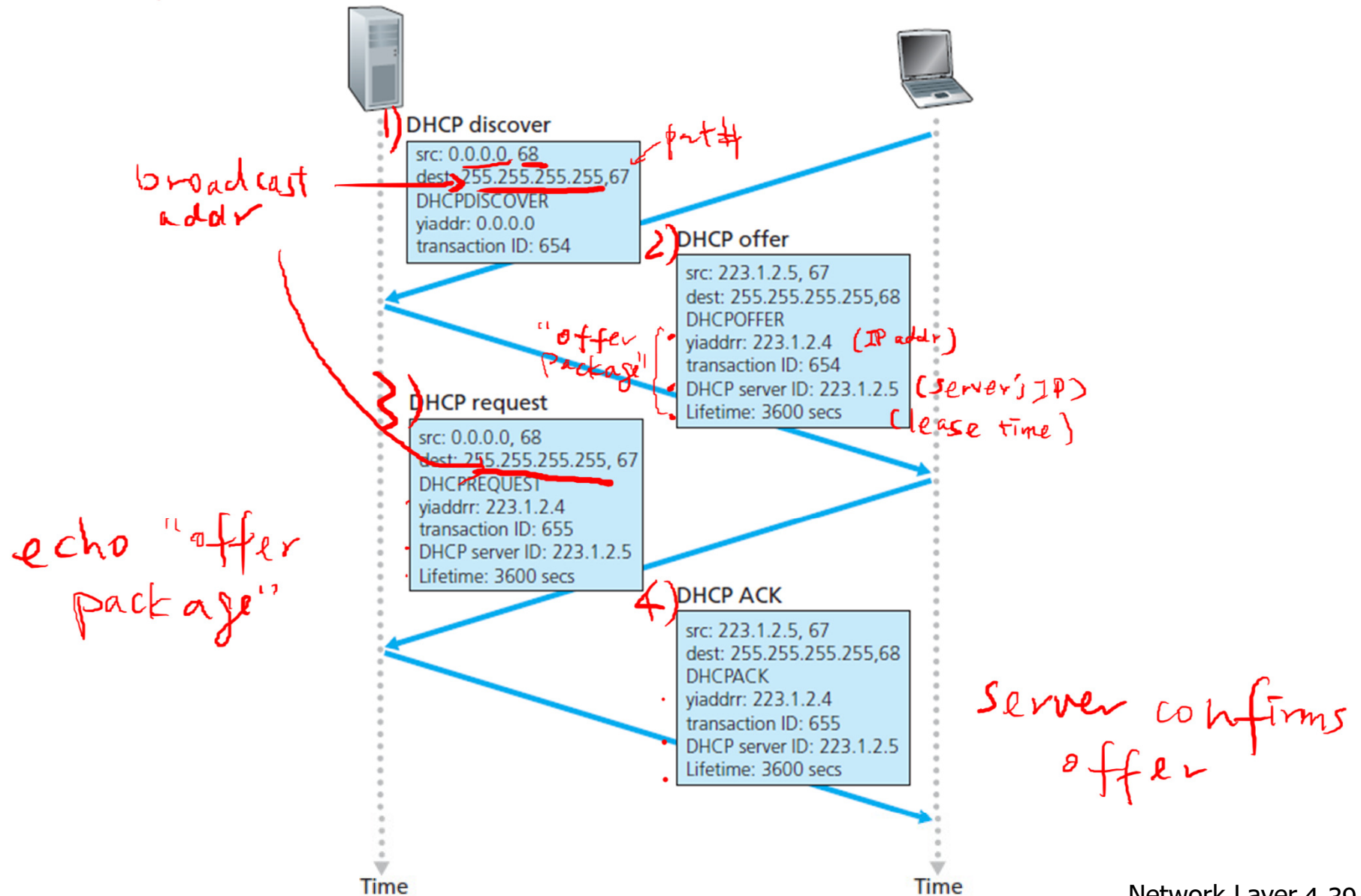
Broadcast: OK. You've
got that IP address!

DHCP client-server scenario

UDP (transport layer)

DHCP server:
223.1.2.5

Arriving client

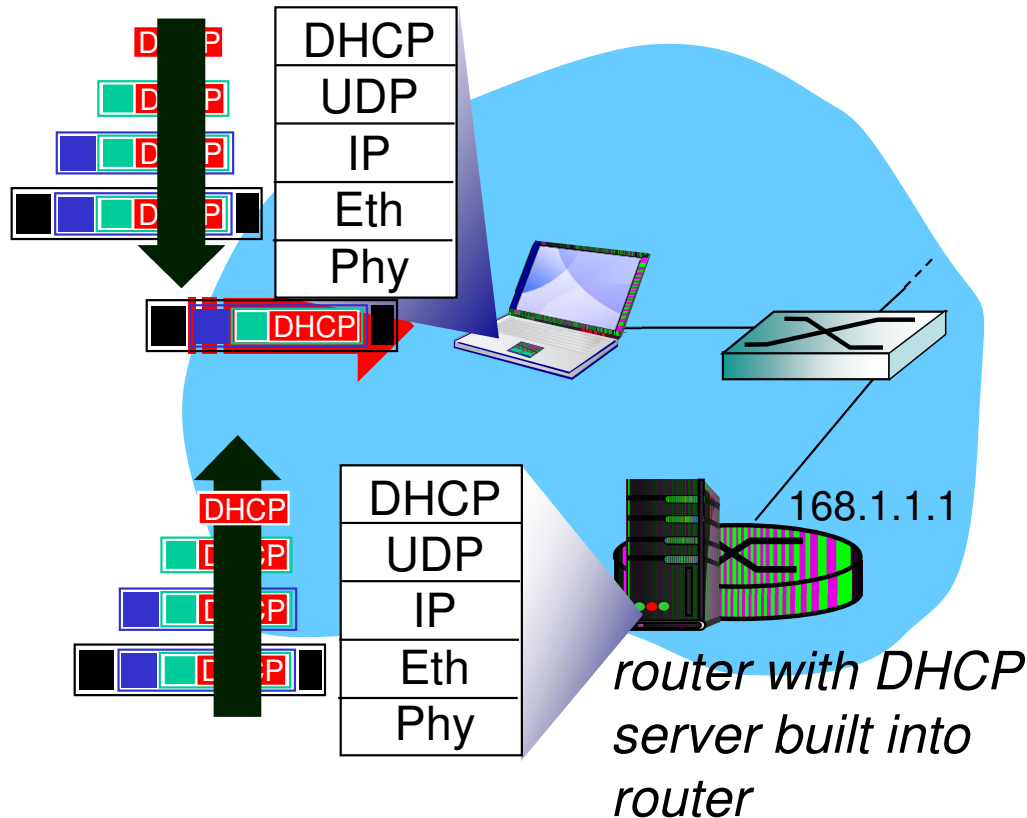


DHCP: more than IP addresses

DHCP can return more than just allocated IP address on subnet:

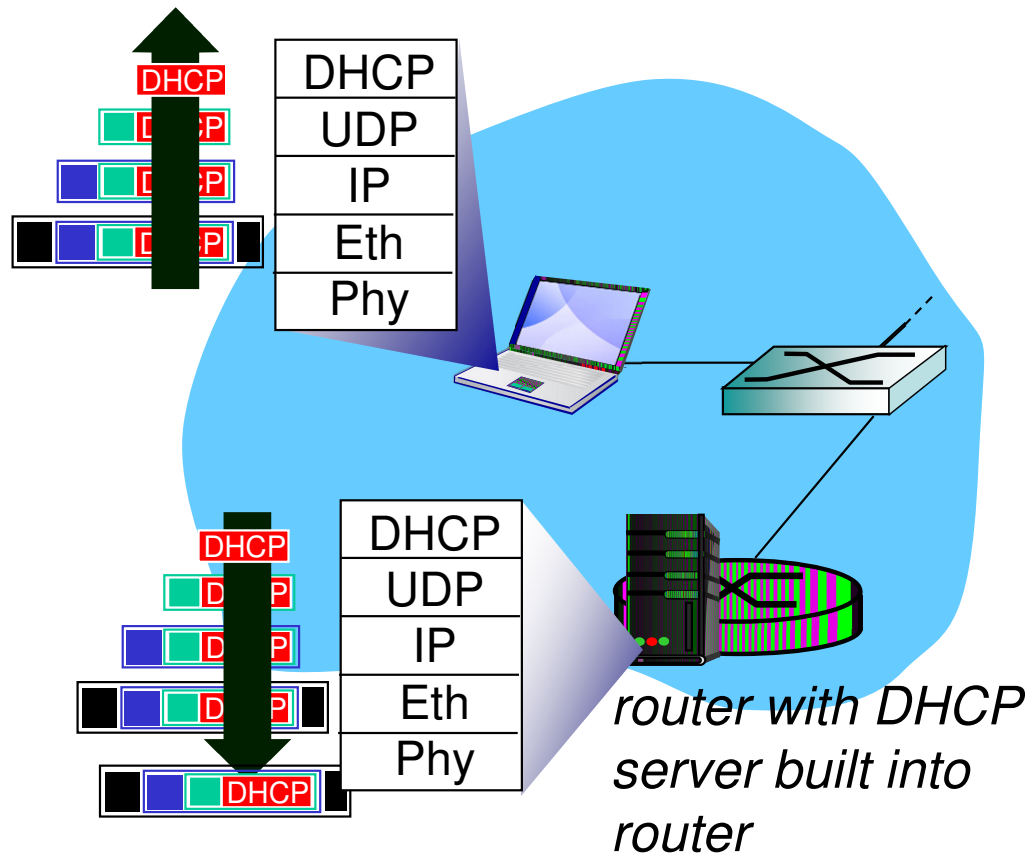
- address of first-hop router for client
- name and IP address of DNS sever
- network mask (indicating network versus host portion of address)

DHCP: example



- ❖ connecting laptop needs its IP address, addr of first-hop router, addr of DNS server: use DHCP
- ❖ DHCP request encapsulated in UDP, encapsulated in IP, encapsulated in 802.1 Ethernet
- ❖ Ethernet frame broadcast (dest: FFFFFFFFFFFF) on LAN, received at router running DHCP server
- ❖ Ethernet demuxed to IP demuxed, UDP demuxed to DHCP

DHCP: example



- ❖ DCP server formulates DHCP ACK containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- ❖ encapsulation of DHCP server, frame forwarded to client, demuxing up to DHCP at client
- ❖ client now knows its IP address, name and IP address of DSN server, IP address of its first-hop router

DHCP: Wireshark output (home LAN)

Message type: Boot Request (1)

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 0.0.0.0 (0.0.0.0)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 0.0.0.0 (0.0.0.0)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Server host name not given

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) **DHCP Message Type = DHCP Request**

Option: (61) Client identifier

Length: 7; Value: 010016D323688A;

Hardware type: Ethernet

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Option: (t=50,l=4) Requested IP Address = 192.168.1.101

Option: (t=12,l=5) Host Name = "nomad"

Option: (55) Parameter Request List

Length: 11; Value: 010F03062C2E2F1F21F92B

1 = Subnet Mask; 15 = Domain Name

3 = Router; 6 = Domain Name Server

44 = NetBIOS over TCP/IP Name Server

.....

request

Message type: Boot Reply (2)

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 192.168.1.101 (192.168.1.101)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 192.168.1.1 (192.168.1.1)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Server host name not given

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) DHCP Message Type = DHCP ACK

Option: (t=54,l=4) Server Identifier = 192.168.1.1

Option: (t=1,l=4) Subnet Mask = 255.255.255.0

Option: (t=3,l=4) Router = 192.168.1.1

Option: (6) Domain Name Server

Length: 12; Value: 445747E2445749F244574092;

IP Address: 68.87.71.226;

IP Address: 68.87.73.242;

IP Address: 68.87.64.146

Option: (t=15,l=20) Domain Name = "hsd1.ma.comcast.net."

reply

IP addresses: how to get one?

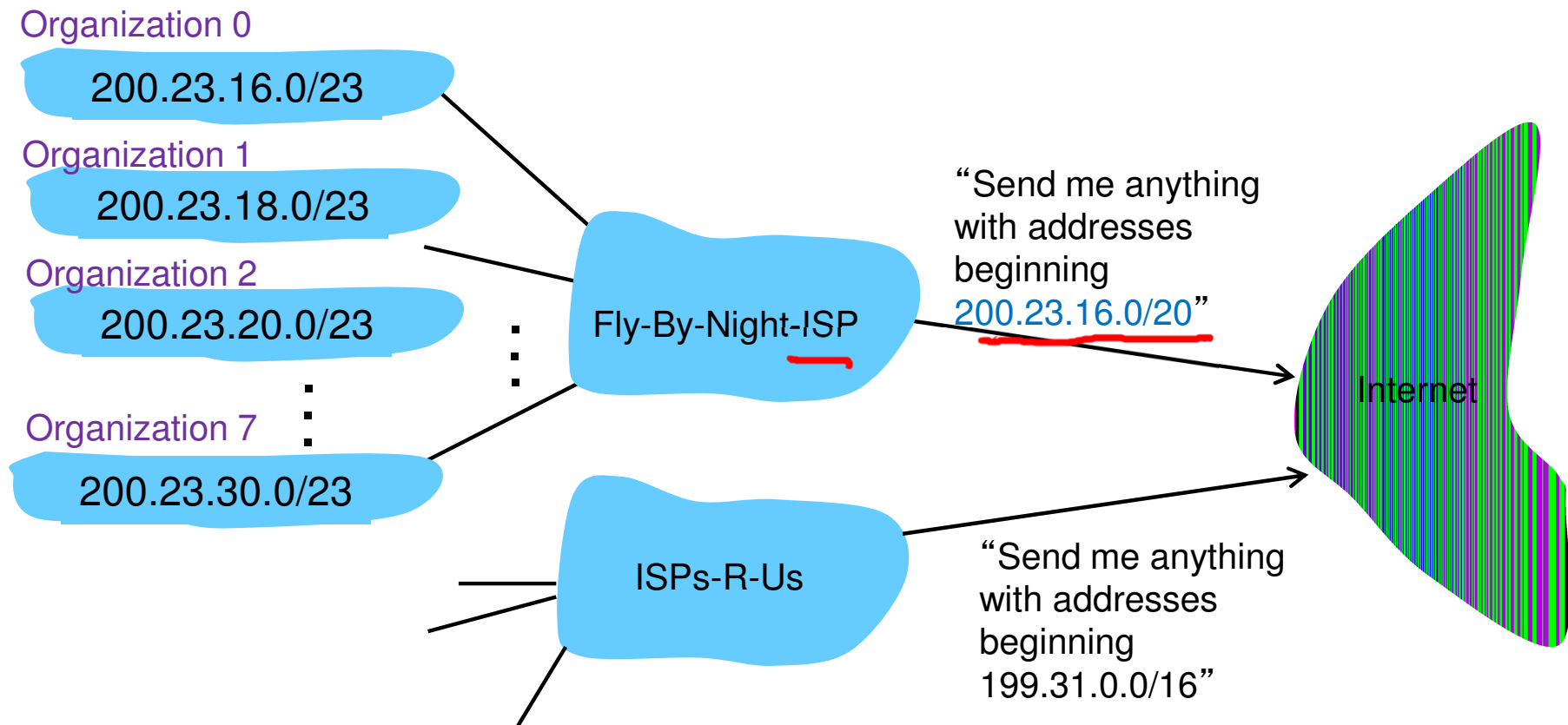
Q: How to obtain a block of IP addresses for use within an organization's subnet?

A: Contact the ISP → provide addresses from a larger block of addresses that had been allocated to it

<u>ISP's block</u>	$\underbrace{11001000\ 00010111\ 0001}_{\text{netw } 26 \text{ bits}} \underbrace{0000}_{\text{to } 0} \underbrace{00000000}_{\text{host 12 bits}}$	200.23.16.0/ <u>20</u>
Organization 0	11001000 00010111 00010000 00000000	200.23.16.0/ <u>23</u>
Organization 1	11001000 00010111 00010010 00000000	200.23.18.0/ <u>23</u>
Organization 2	11001000 00010111 00010100 00000000	200.23.20.0/ <u>23</u>
...		
Organization 7	$\underbrace{11001000\ 00010111\ 00011110}_{\text{netw } 23 \text{ bits}} \underbrace{00000000}_{\text{host 9 bits}}$	200.23.30.0/ <u>23</u> $\begin{matrix} 00 \dots 0 \\ \text{to} \\ 11 \dots 1 \end{matrix} \rightarrow 2^9 - 2 \text{ addr}$

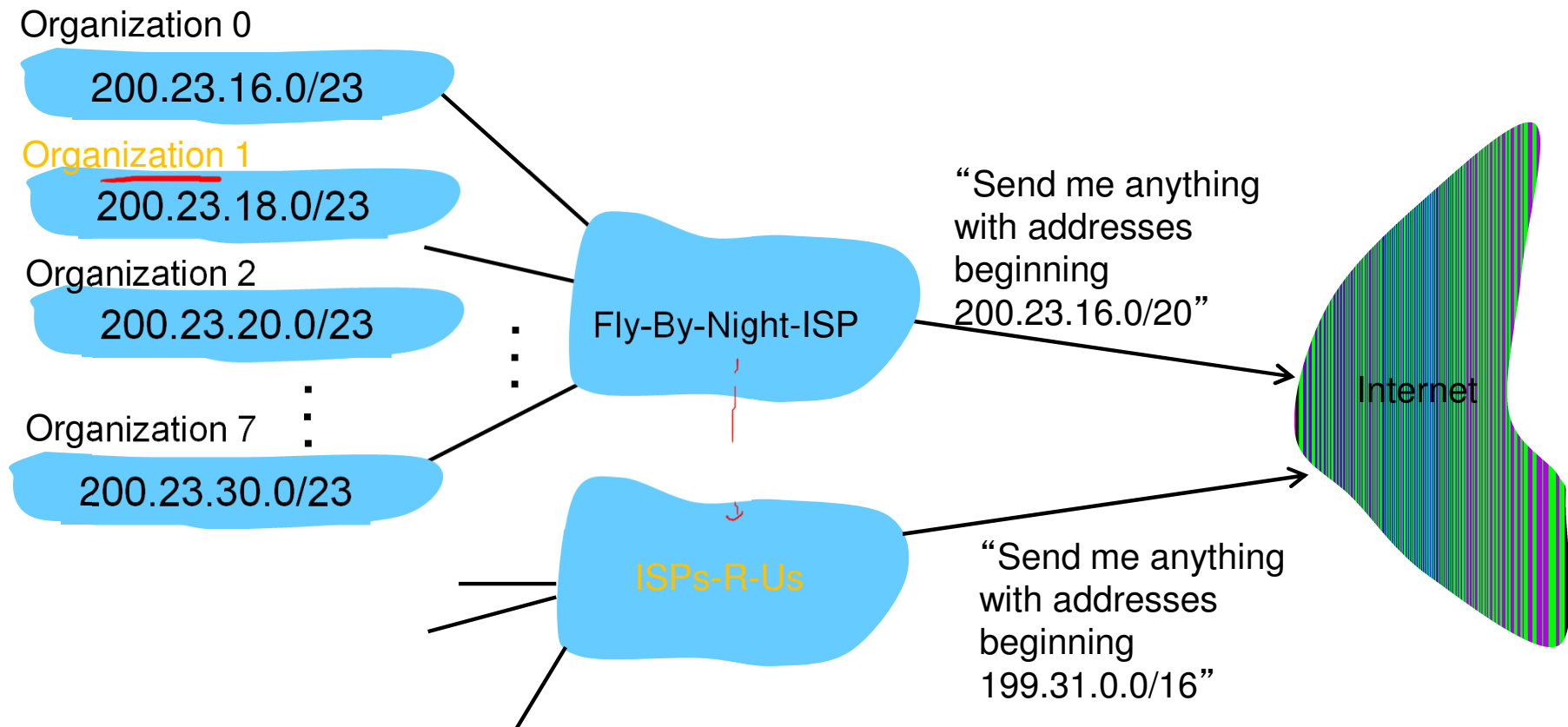
Hierarchical addressing: route aggregation

- Rest of world need not know that within the address block **200.23.16.0/20** there are in fact **eight other organizations**, each with its own subnets.



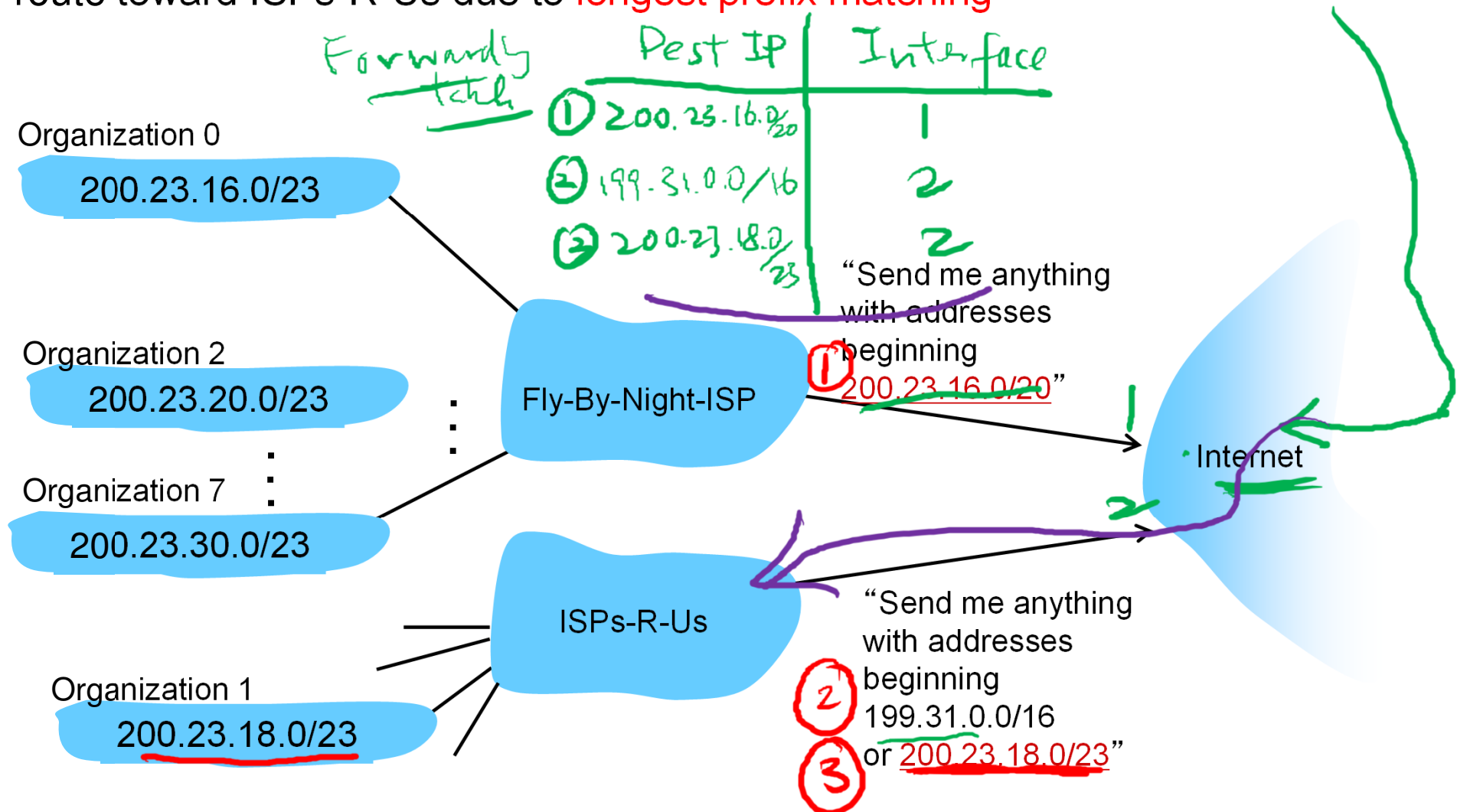
Hierarchical addressing: route aggregation

Q. What would happen if Fly-By-Night-ISP acquires **ISPs-R-U** and then has **Organization 1** connect to the Internet through its subsidiary ISPs-R-U?



Hierarchical addressing: more specific routes

- When other routers in the larger Internet want to route to an address in the block 200.23.18.0/23 for Organization 1
→ route toward ISPs-R-Us due to **longest prefix matching**



Longest Prefix Match

❖ Matching Routing Table Entries

- There can be more than one match due to subnetting or alternative paths
- Always choose the one with the **longest** matching prefix in the network address, i.e., the most specific route
- Example: To route to destination IP 192.168.1.140

IP address (decimal):		192	168	1	140	
IP address (binary):		11000000	10101000	00000001	10001100	
Routing table entries	192.168.1.0/24:	11000000	10101000	00000001	00000000	Match 24 prefix bits
	192.168.1.128/25:	11000000	10101000	00000001	10000000	Match 25 prefix bits
	192.168.0.0/16:	11000000	10101000	00000000	00000000	Match 16 prefix bits

IP addressing: the last word...

Q: how does an ISP get block of addresses?

A: ICANN: Internet Corporation for Assigned Names and Numbers <http://www.icann.org/>

- allocates addresses
- manages DNS
- assigns domain names, resolves disputes

NAT: network address translation

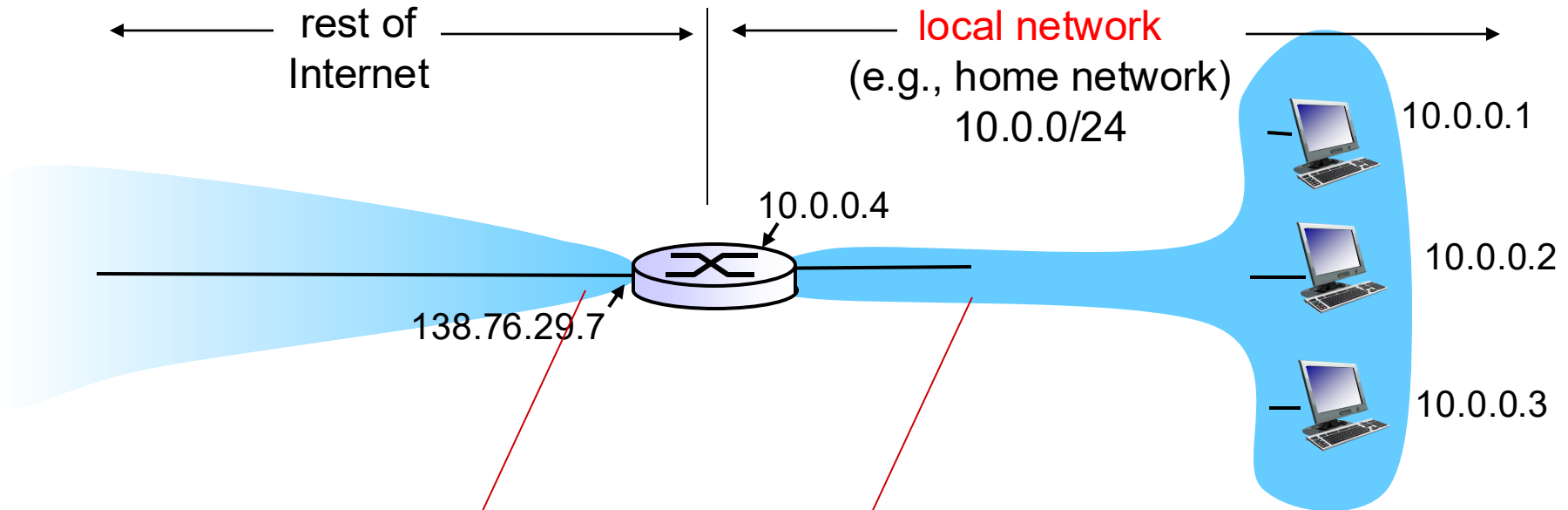
❖ IP Address Shortage:

- IPv4 address space has all been allocated
- In many networks there are more hosts than the number of public IP addresses available
- There are **private** IPv4 addresses (RFC1918)
 - Packets with private IPs are dropped by ISPs

RFC1918 name	IP address range	number of addresses	largest CIDR block (subnet mask)	host id size	mask bits
24-bit block	10.0.0.0 - 10.255.255.255	16,777,216	10.0.0.0/8 (255.0.0.0)	24 bits	8 bits
20-bit block	172.16.0.0 - 172.31.255.255	1,048,576	172.16.0.0/12 (255.240.0.0)	20 bits	12 bits
16-bit block	192.168.0.0 - 192.168.255.255	65,536	192.168.0.0/16 (255.255.0.0)	16 bits	16 bits

Source: http://en.wikipedia.org/wiki/Private_network#Private_IPv4_address_spaces

NAT: network address translation



all datagrams *leaving* local network have:
-**same** single source NAT IP address: 138.76.29.7,
-**different** source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT: network address translation

motivation: local network uses just one IP address as far as outside world is concerned:

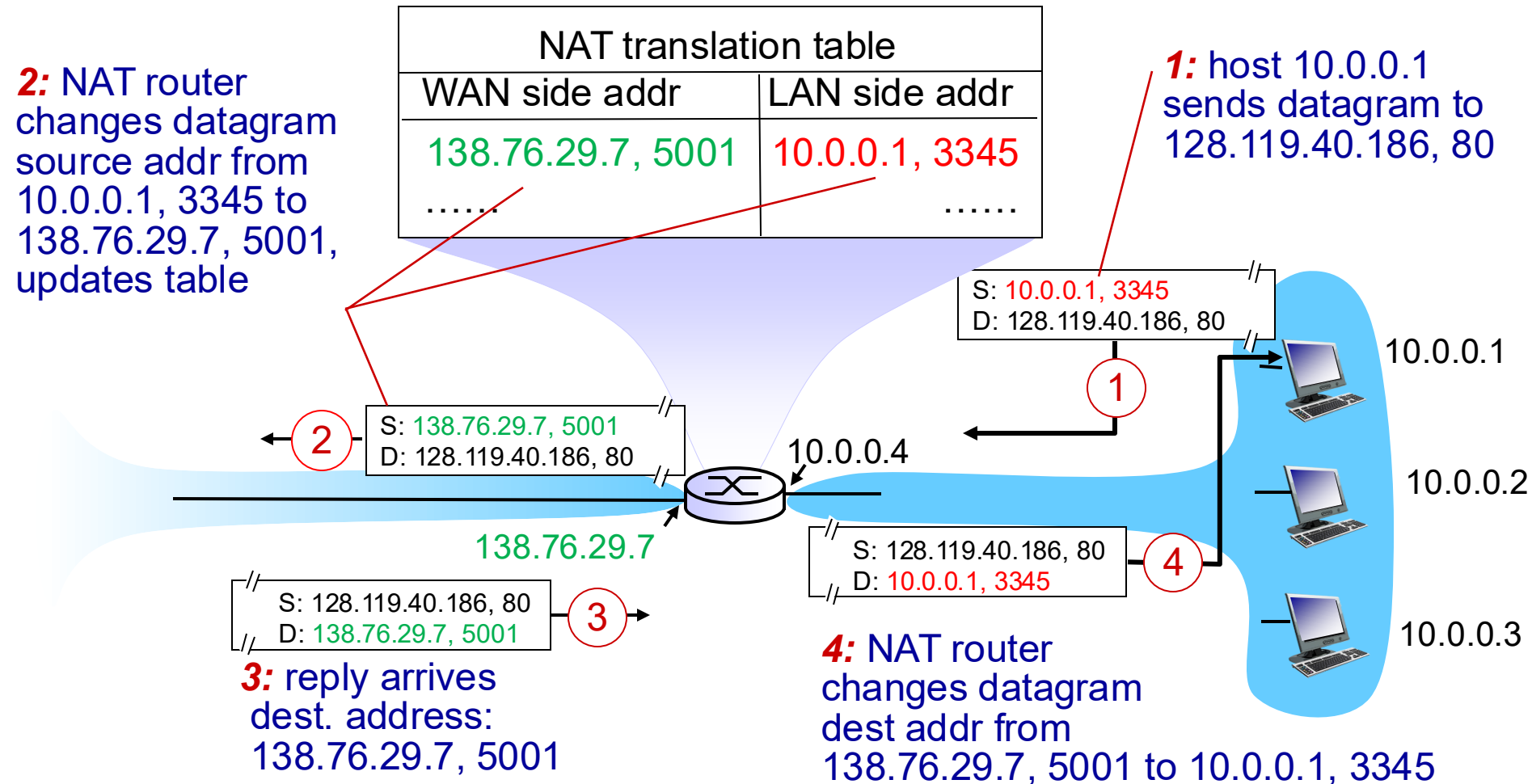
- range of addresses not needed from ISP: just one IP address for all devices
- can change addresses of devices in local network without notifying outside world
- can change ISP without changing addresses of devices in local network
- devices inside local net not explicitly addressable, visible by outside world (a security plus)

NAT: network address translation

implementation: NAT router must:

- remember in NAT translation table every (source IP address, port #) to (NAT IP address, new port #) translation pair
- *outgoing datagrams:* replace (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)
... remote clients/servers will respond using (NAT IP address, new port #) as destination addr
- *incoming datagrams:* replace (NAT IP address, new port #) in dest fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

NAT: network address translation



NAT: network address translation

- ❖ 16-bit port-number field:
 - 60,000 simultaneous connections with a single LAN-side address!
- ❖ NAT is **controversial**:
 - routers should only process up to layer 3
 - violates end-to-end argument
 - NAT possibility must be taken into account by app designers, e.g., **P2P applications**
 - address shortage should instead be solved by IPv6

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

IPv6: motivation

- ❖ *initial motivation*: 32-bit address space soon to be completely allocated.
- ❖ additional motivation:
 - header format helps speed processing/forwarding
 - header changes to facilitate quality of service (QoS)

IPv6 datagram format:

- fixed-length 40 byte header
- no fragmentation allowed
- Expanded addressing capabilities:
 - 32 bits (IPv4) → 128 bits (IPv6)

IPv6 Network Addresses

❖ IPv6 Address

- **128 bits** – no more IP address shortage!
- Representation: 8 sets of 2-byte hexadecimal blocks

2041:0000:140F:0000:0000:0000:005B:131B

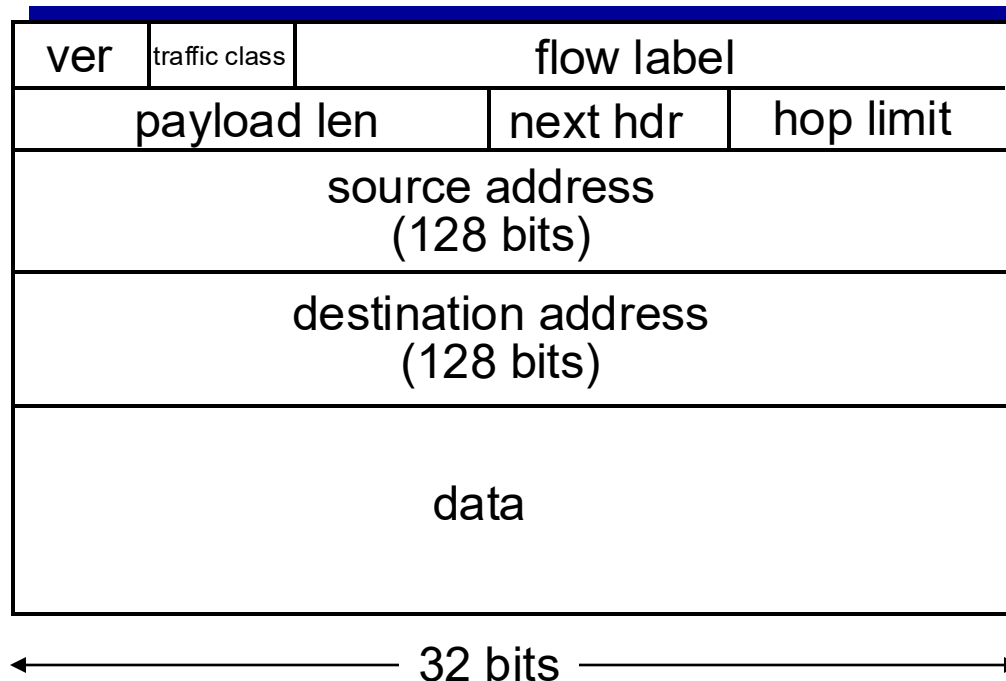
↑
20_H=00100000_B

- Address space:
340,282,366,920,938,463,463,374,607,431,768,211,456
- In progress of roll-out, mostly regional or within an organization/ISP
- Google measured ~44.98% IPv6 users in Nov 2025, see <http://www.google.com/intl/en/ipv6/statistics.html>.
- **Not compatible with IPv4**

IPv6 datagram format

traffic class: identify priority among datagrams in flow

next header: identify upper layer protocol for data

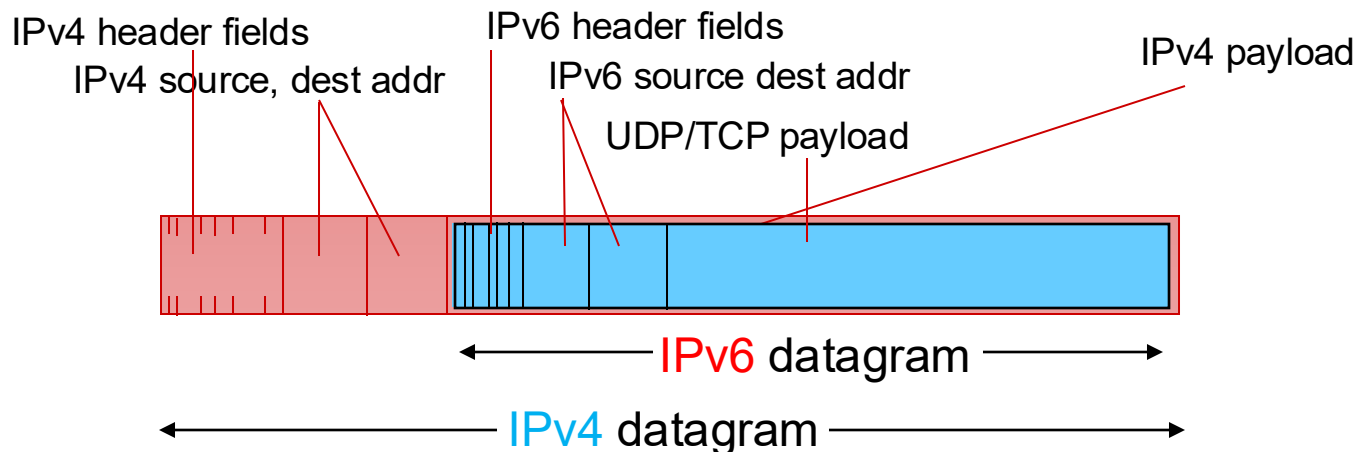


Other changes from IPv4

- ❖ *ICMPv6*: new version of ICMP
 - additional message types, e.g. “Packet Too Big”
 - multicast group management functions
- ❖ *checksum*: removed entirely to reduce processing time at each hop

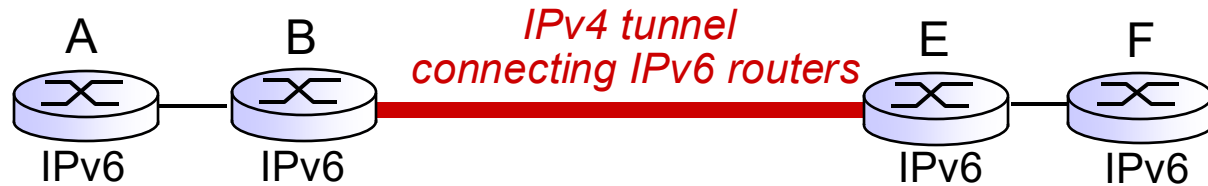
Transition from IPv4 to IPv6

- ❖ not all routers can be upgraded simultaneously
 - no “flag days” (turn off all machines and upgrade)
- ❖ **Q.** How will network operate with **mixed** IPv4 and IPv6 routers?
- ❖ **Tunneling:** IPv6 datagram carried as **payload** in IPv4 datagram among IPv4 routers

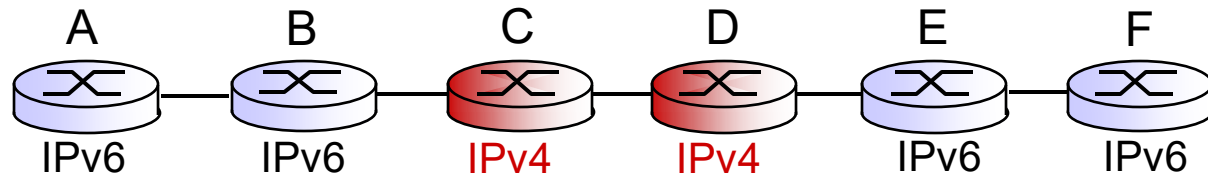


Tunneling

logical view:

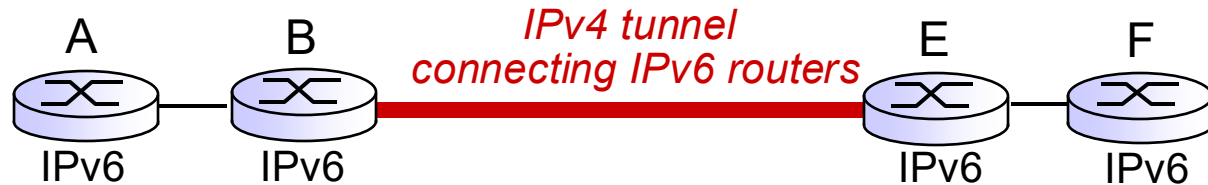


physical view:

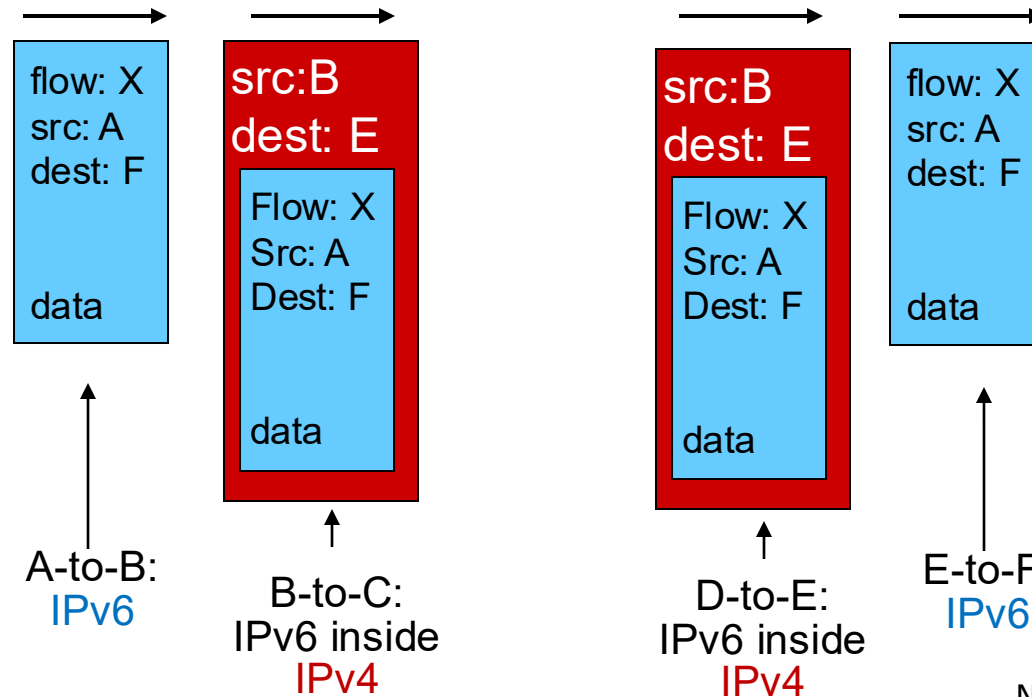
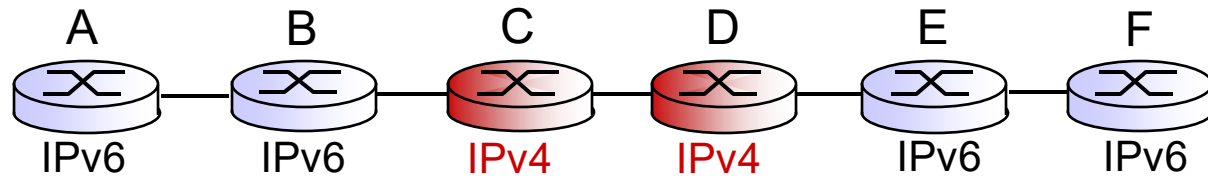


Tunneling

logical view:



physical view:



Chapter 4: done!

4.1 Overview of Network layer: data plane and control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- NAT
- IPv6

4.4 Generalized Forward and SDN

- match plus action
- OpenFlow example

Question: how do forwarding tables (destination-based forwarding) or flow tables (generalized forwarding) computed?

Answer: by the control plane (next chapter)